

Project - 4

Group Members: Mounika Boggavarapu
Subhasya Tippareddy
Sri Harsha Mulluri

Operating System - Windows 11 - (WSL) Ubuntu 22.04.3 LTS Release - 22.04
Processor - Intel i7 11th Gen
RAM - 16 GB

PART 1: Adding lseek support

lseek function moves the file pointer across the file.

We implemented a system call – 'lseek' which takes file descriptor and offset as parameters.

As instructed, we assumed the functionality of the SEEK_CUR, repositioning the offset to the given byte relative to the current file descriptor offset location.

The system call implemented in sysfile.c verifies the validity of the file descriptor and increases the file pointer by the offset received as parameter. When the file offset is less than zero, we set it back to zero to avoid errors. If the file pointer is moved beyond f->ip->size, we fill the gaps with 0 as instructed. We modified filewrite function of file.c to write zeroes into the file when f->off > f->ip->size.

Userfile – lseek_test.c

To test the functionality of lseek we've implemented, we came up with a user test. This program starts with creating a file – lseektest.txt and writes 'Deadpool' into it. We call the lseek function passing the file descriptor and 15 as the offset parameter. This should result in 15 zeroes written after 'Deadpool'.

Now we write the phrase – 'Wolverine - Hero' into lseektest.txt. We again call lseek function, this time passing -6 as offset parameter, and write ' - Villain' into the file. The final content in the file should be

'Deadpool0000000000000000Wolverine - Villain'.

```
gcc -fno-pie -static -fno-builtin -fno-strict-aliasing -O2 -Wall -R0 -ggdb -m32 -Werror -fno-omit-frame-pointer -fno-stack-protector -fno-pie -no-pie -c -o lseektest.o lseektest.c
ld -m elf_i386 -M -e main -Ttext 0 -o _lseektest lseektest.o ulib.o usys.o printf.o umalloc.o
objdump -S _lseektest > lseektest.asm
objdump -t _lseektest | sed '1,/SYMBOL TABLE/d; s/ .* //; /$/d' > lseektest.sym
./mifs fs.img sampletext.txt README _cat _echo _forktest _grep _ls _init _kill _ln _mkdir _find _rm _sh _stressfs _usertests _sleep _wc _zombie _uniq _lseek
test _largetest 1 _largetest 2
mmeta 120 (boot, super, log blocks 30 inode blocks 26, bitmap blocks 62) blocks 249880 total 250000
ballocc: first 910 blocks have been allocated
ballocc: write bitmap block at sector 58
qemu-system-i386 -serial mon:stdio -drive file=fs.img,index=1,media=disk,format=raw -drive file=xv6.img,index=0,media=disk,format=raw -smp 2 -m 512
xv6.
cpu0: starting 0
sb: size 250000 nblocks 249880 ninodes 200 nlog 30 logstart 2 inodestart 32 bmap start 58
init: starting sh
$ lseektest
Testing lseek functionality:
Increasing the Offset by 15
Decreasing Offset by -6
$ cat lseektest.txt
Deadpool0000000000000000Wolverine - Villain$ |
```

PART 2: Symbolic Links

We have successfully implemented the symbolic links by enhancing the system's file management capabilities. This was achieved by integrating a new system call symlink, which we developed in sysfile.c, syscall.c, syscall.h, usys.S, stat.h, user.h. Here's how we did it:

First, we modified the create function in sysfile.c to handle the creation of symbolic link type files (T_SYMLINK). When symlink is invoked, it calls create to allocate a new inode, which is then used to store the path of the target file. And we modified the open system call in sysfile.c which will take the inode pointer from the create system call, and check how many times it is going through the symlink and also check the type of the link whether it is a symlink or not.

```
$ symlinktest
Symbolic link 'outerlink.txt' created pointing to 'innerlink.txt'
```

And the path of the link is written directly into the inode's data blocks, setting up the inode to act as a pointer to another file location and this is done by the symlink system call, if the file was not read correctly then it will generate an error. We created symlinktest.c user file in which we will call the symlink system call function, here are creating the symlink for some existing files and non existing files and also the cyclelink file in which all links are cycle and file never exist. We specifically ensured that this setup can handle links that point to non-existent files.

```
$ cat nonexistent.txt
cat: cannot open nonexistent.txt
```

To address the issue of recursive or cyclic links, which can cause system crashes or infinite loops, we added a depth control mechanism. Our implementation checks the depth of symbolic links up to a maximum of 10 levels, this can ensure that we do not go into trap or infinite loop links.

```
$ cat cyclelink1.txt
cat: cannot open cyclelink1.txt
$ cat cyclelink2.txt
cat: cannot open cyclelink2.txt
$ cat cyclelink3.txt
cat: cannot open cyclelink3.txt
```

If this depth is exceeded, the operation is safely aborted. When running ls, the output clearly shows all the files and links created, including those involved in cyclic or non-existent path scenarios.

```
$ ls
sampletext.txt 2 2 209
README         2 3 2286
cat            2 4 15748
echo           2 5 14628
forktest       2 6 9076
grep           2 7 18592
ls             2 8 18004
init           2 9 15248
kill           2 10 14712
ln             2 11 14608
mkdir          2 12 14736
find           2 13 24204
rm             2 14 14716
sh             2 15 28776
stressfs       2 16 15644
usertests      2 17 63148
sleep          2 18 14764
wc             2 19 16172
zombie         2 20 14292
uniq           2 21 19800
symlinktest    2 22 16700
clonetest      2 23 16240
locktest       2 24 17536
console        3 25 0
innerlink.txt  2 26 37
outerlink.txt  2 27 17
mainlink.txt   2 28 44
unable to open ./nonexist.txt
unable to open ./cyclelink2.txt
unable to open ./cyclelink3.txt
unable to open ./cyclelink4.txt
unable to open ./cyclelink5.txt
unable to open ./cyclelink6.txt
unable to open ./cyclelink1.txt
```

PART 3: Large files

As given in instructions, we changed FSSIZE value to 250000 in param.h .

The current file system only allows files of 140 blocks.

We write a user program - largeftest_1.c, which writes 16,523 blocks into a file. This will not run successfully.

```
dd if=/dev/zero of=xv6.img count=10000
10000+0 records in
10000+0 records out
5120000 bytes (5.1 MB, 4.9 MiB) copied, 2.19026 s, 2.3 MB/s
dd if=bootblock of=xv6.img conv=notrunc
1+0 records in
1+0 records out
512 bytes copied, 0.00571879 s, 89.5 kB/s
dd if=kernel of=xv6.img seek=1 conv=notrunc
393+1 records in
393+1 records out
201588 bytes (202 kB, 197 KiB) copied, 0.205007 s, 983 kB/s
qemu-system-i386 -serial mon:stdio -drive file=fs.img,index=1,media=disk,format=raw -drive file=xv6.img,index=0,media=disk,format=raw -smp 2 -m 512
xv6...
cpu0: starting 0
sb: size 250000 nblocks 249880 ninodes 200 nlog 30 logstart 2 inodestart 32 bmap start 58
init: starting sh
$ largeftest_1
Error - Unable to write blocks as requested into the file.
```

To support files of 16,523 blocks, we use a double indirect block in inode. Firstly, we use only 11 direct blocks instead of 12 and defined the double indirect block in inode data structure as NDINDIRECT (NINDIRECT * NINDIRECT) and modify MAXFILE. To implement the block, we modified the bmap function of fs.c file.

Now, if we run largeftest_1.c, we see that a file of 16,523 blocks is created successfully.

```
gcc -fno-pic -static -fno-built-in -fno-strict-aliasing -O2 -Wall -MD -ggdb -m32 -Werror -fno-omit-frame-pointer -fno-stack-protector -fno-pie -no-pie -c -o largeftest_1.o largeftest_1.c
ld -m elf_i386 -N -e main -Text 0 -o _largeftest_1 largeftest_1.o ulib.o usys.o printf.o umalloc.o
objdump -S _largeftest_1 > largeftest_1.asm
objdump -t _largeftest_1 | sed '1,/SYMBOL TABLE/d; s/ .* / /; /^$/d' > largeftest_1.sym
./mkfs fs.img sampletext.txt README _cat _echo _forktest _grep _ls _init _kill _ln _mkdir _find _rm _sh _stressfs _usertests _sleep _wc _zombie _uniq _largeftest_1
nmeta 120 (boot, super, log blocks 30 inode blocks 26, bmap blocks 62) blocks 249880 total 250000
balloc: first 856 blocks have been allocated
balloc: write bitmap block at sector 58
qemu-system-i386 -serial mon:stdio -drive file=fs.img,index=1,media=disk,format=raw -drive file=xv6.img,index=0,media=disk,format=raw -smp 2 -m 512
xv6...
cpu0: starting 0
sb: size 250000 nblocks 249880 ninodes 200 nlog 30 logstart 2 inodestart 32 bmap start 58
init: starting sh
$ largeftest_1
A file by the name - largeftest_1 is created of 16,523 blocks
```

We wrote a second user program - largeftest_2.c , which writes 32,906 blocks into a file. This will not run successfully.

```
xv6...
cpu0: starting 0
sb: size 250000 nblocks 249880 ninodes 200 nlog 30 logstart 2 inodestart 32 bmap start 58
init: starting sh
$ largeftest_2
A file by the name - largeftest_2 is created of 32,906 blocks
```

To support files of 32,906 blocks, we implement a second double indirect block in inode. Again, we decreased direct blocks from 11 to 10 and modified the MAXFILE. We implemented the second doubly indirect block in the bmap function of fs.c file.

Now, if we run largeftest_2.c, we see that a file of 32,906 blocks is created successfully.

We modify largeftest_2.c to write 32,907 blocks into the file and it fails to write as expected.

To make sure we are freeing the double indirect blocks , we modified the itrunc function of fs.c. This will free the blocks of the file. Following screenshot shows the freeing of blocks when file is deleted without any issues.

```

$ ls
sampletext.txt 2 2 209
README         2 3 2286
cat            2 4 15576
echo           2 5 14460
forktest       2 6 8904
grep           2 7 18420
ls             2 8 17836
init           2 9 15080
kill           2 10 14544
ln             2 11 14440
mkdir          2 12 14568
find           2 13 24036
rm             2 14 14548
sh             2 15 28604
stressfs       2 16 15476
usertests      2 17 62976
sleep          2 18 14596
wc             2 19 16004
zombie         2 20 14124
uniq           2 21 19632
largeftest_1   2 22 14972
largeftest_2   2 23 14972
console        3 24 0
largeftest_1.t 2 25 8459776
largeftest_2.t 2 26 16847872
$ rm largeftest_1.txt
$ rm largeftest_2.txt
$

```

PART 4: Extent based file system

To implement the extent based file system, we have not changed the inode structure (same as PART-3). We are only adding extent based support for direct pointers. We have 10 direct pointers (NDIRECT), each of 4 bytes. We have decided to allocate 1 byte for length and 3-bytes for pointer.

Keeping the pointer-based file system as it is, we have added extent-based using a flag `O_EXTENT` (defined in `fcntl.h`) for creation, in `open()` system call. In `stat.h`, we have added `T_EXTENT`, which is another file type and also added a new struct `file_extent` which has pointer and length. Then, we modify the `bmap()` function to add support for extent based files.

Implementation -

We check if the block number is within the range for the direct extents, $256 \times \text{NDIRECT}$.

If yes, We go through the direct extents (up to `NDIRECT`) and calculate the cumulative length of extents processed so far (`curr_len`). If the block number falls within the current extent, it computes the actual disk address by adding the offset within the extent to the extent's start address and returns it.

If the address for the block number is not found, we allocate a new block (`ballocc`) and then check if this newly allocated bloc can be appended to an existing extent (i.e., if it is contiguous with the last block of an existing extent). If yes, we update the extent length. If not contiguous, we look for an empty extent slot and assign the address to it, shifting it by 8 bits (storing the length in the lower byte). If no empty extent slot is found, we free the allocated block (`bfree`). If the block number is outside the range of what the extents can cover, it triggers a panic with the message "bmap: out of range".

Testing -

We have created 2 user programs, file_extent_test.c and stat.c

<filename> has to be the same in all places.

To test creation and write operation -

Command: file_extent_test <filename>

```
sb: size 250000 nblocks 249880 ninodes 200 nlog 30 logstart 2 inodestart 32 bmap start 58
```

```
init: starting sh
```

```
$ file_extent_test sample.txt
```

```
sample.txt created successfully
```

```
Empty address added at block: 1
```

```
Address 1020 added to block: 1, of length: 1
```

```
Address 1020 added to block: 1, of length: 2
```

```
Address 1020 added to block: 1, of length: 3
```

```
Address 1020 added to block: 1, of length: 4
```

```
Address 1020 added to block: 1, of length: 5
```

```
Address 1020 added to block: 1, of length: 6
```

```
Address 1020 added to block: 1, of length: 7
```

```
Address 1020 added to block: 1, of length: 8
```

```
Address 1020 added to block: 1, of length: 9
```

```
Address 1020 added to block: 1, of length: 10
```

```
Address 1020 added to block: 1, of length: 11
```

```
Address 1020 added to block: 1, of length: 12
```

```
Address 1020 added to block: 1, of length: 13
```

```
Address 3324 added to block: 10, of length: 241
```

```
Address 3324 added to block: 10, of length: 242
```

```
Address 3324 added to block: 10, of length: 243
```

```
Address 3324 added to block: 10, of length: 244
```

```
Address 3324 added to block: 10, of length: 245
```

```
Address 3324 added to block: 10, of length: 246
```

```
Address 3324 added to block: 10, of length: 247
```

```
Address 3324 added to block: 10, of length: 248
```

```
Address 3324 added to block: 10, of length: 249
```

```
Address 3324 added to block: 10, of length: 250
```

```
Address 3324 added to block: 10, of length: 251
```

```
Address 3324 added to block: 10, of length: 252
```

```
Address 3324 added to block: 10, of length: 253
```

```
Address 3324 added to block: 10, of length: 254
```

```
Address 3324 added to block: 10, of length: 255
```

Ls: sample.txt has been created successfully (at the last)

```
$ ls
sampletext.txt 2 2 201
README         2 3 2286
cat            2 4 15672
echo           2 5 14552
forktest       2 6 8996
grep           2 7 18516
ls             2 8 18016
init           2 9 15176
kill           2 10 14636
ln             2 11 14536
mkdir          2 12 14660
find           2 13 24376
rm             2 14 14644
sh            2 15 28700
stressfs       2 16 15568
usertests      2 17 63072
sleep         2 18 14688
wc            2 19 16096
zombie         2 20 14220
uniq           2 21 19728
lseek_test     2 22 14820
symlink_test   2 23 16628
largeftest_1   2 24 15072
largeftest_2   2 25 15064
stat           2 26 15440
file_extent_te 2 27 15384
console        3 28 0
sample.txt     5 29 1310720
```

To read the file and display file information -

Command: stat <filename>

For extent_based file:

```
$ stat sample.txt
type: It is an extent-based file
size: 1310720
Extent Block: 1 Start Pointer Address: 1020 Length: 255
Extent Block: 2 Start Pointer Address: 1276 Length: 255
Extent Block: 3 Start Pointer Address: 1532 Length: 255
Extent Block: 4 Start Pointer Address: 1788 Length: 255
Extent Block: 5 Start Pointer Address: 2044 Length: 255
Extent Block: 6 Start Pointer Address: 2300 Length: 255
Extent Block: 7 Start Pointer Address: 2556 Length: 255
Extent Block: 8 Start Pointer Address: 2812 Length: 255
Extent Block: 9 Start Pointer Address: 3068 Length: 255
Extent Block: 10 Start Pointer Address: 3324 Length: 255
```

For pointer based file:

```
$ stat sampletext.txt
type: It is a pointer-based file
size: 201
```

To delete the file -

Command rm <filename>

Following screenshot shows successful deletion of a pointer based and an extent based file.

```
$ rm sample.txt
$ rm sampletext.txt
$ ls
README          2  3 2286
cat              2  4 15672
echo            2  5 14552
forktest        2  6  8996
grep            2  7 18516
ls              2  8 18016
init            2  9 15176
kill            2 10 14636
ln              2 11 14536
mkdir           2 12 14660
find            2 13 24376
rm              2 14 14644
sh              2 15 28700
stressfs        2 16 15568
usertests       2 17 63072
sleep           2 18 14688
wc              2 19 16096
zombie          2 20 14220
uniq            2 21 19728
lseek_test      2 22 14820
symlink_test    2 23 16628
largeftest_1    2 24 15072
largeftest_2    2 25 15064
stat            2 26 15440
file_extent_te  2 27 15384
console         3 28  0
```